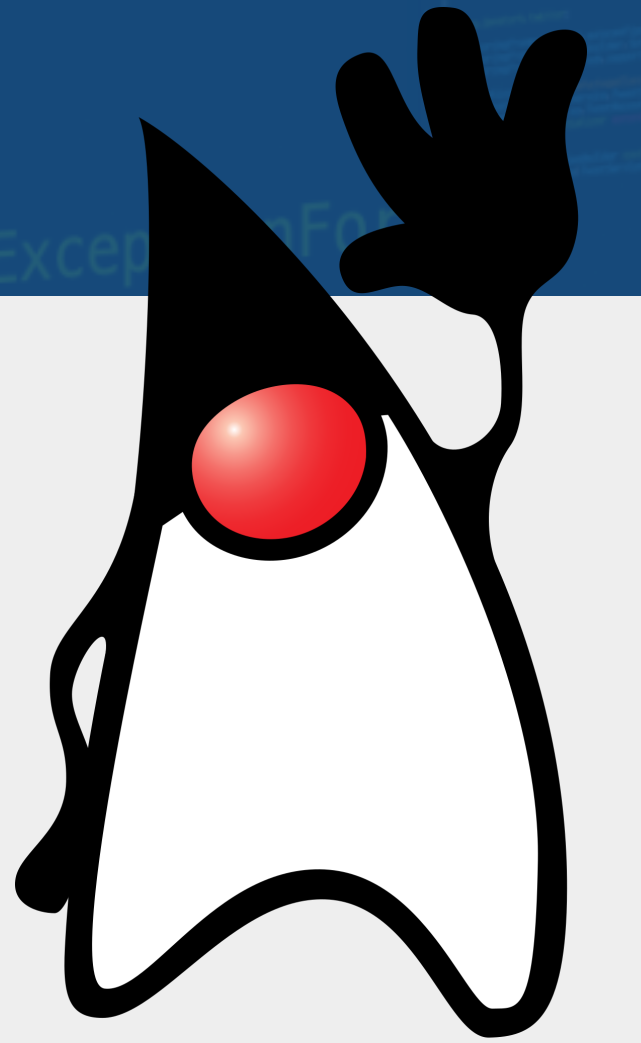


Java Programlama Dili

20. Bölüm

Bellek Yönetimi

Akın Kaldıroğlu



Amaç



- Bu bölümde bellek yönetimine giriş yapılacaktır



- **Bellek Terimleri**
 - Bellek Sızıntısı
- **Bellek Davranışı**
 - Etkin Bellek Yönetimi
 - Garbage Collection
- **Ödevler**
- **Bundan Sonra?**

Bellek Terimleri



Çalışma Zamanı Bellekleri



- Şu ana kadar Java'nın çalışma zamanında kullandığı üç farklı bellek yapısından bahsedildi:
- **Stack**: Metotların çalışması için gerekli olan bellek, yerel değişkenlerin bulunduğu alandır
- **Heap**: Nesnelerin ve yüklenen sınıfların yaşadığı bellek
- **Metaspace**: Sınıfların, bytecode, sabit havuzu, alan/metot tanımları vb. metaverisinin tutulduğu bellek
- Java 8 öncesindeki adı: **PermGen (Permanent Generation)**

Çöp Toplama (Garbage Collection)



- Nesne merkezli dillerle geliştirilen uygulamalar genel olarak yoğun nesne yaratır, dolayısıyla da heap yoğun kullanır
- Bu yüzden heap alanının yönetimi önemlidir
- Kullanılmayan nesnelerin bellekte işgal ettiği alanların, yeni nesneler için kullanılabilir hale getirilmesine, bir başka deyişle, nesnelerin bellekten temizlenmesine, **Çöp Toplama (Garbage Collection, GC)** denir

Çöp Toplayıcı (Garbage Collector)



- Java'da nesnelerin oluşturulması, bellekten silinmesi ve bunlarla ilgili **bellek organizasyonunun (memory organization)** yönetilmesinden sorumlu olan yapıya **Çöp Toplayıcı (Garbage Collector, GC)** denir
- GC, JVM'in içinde sürekli çalışan yapılardandır ve heap bellek alanının yönetiminden sorumludur

Otomatik Bellek Yönetimi



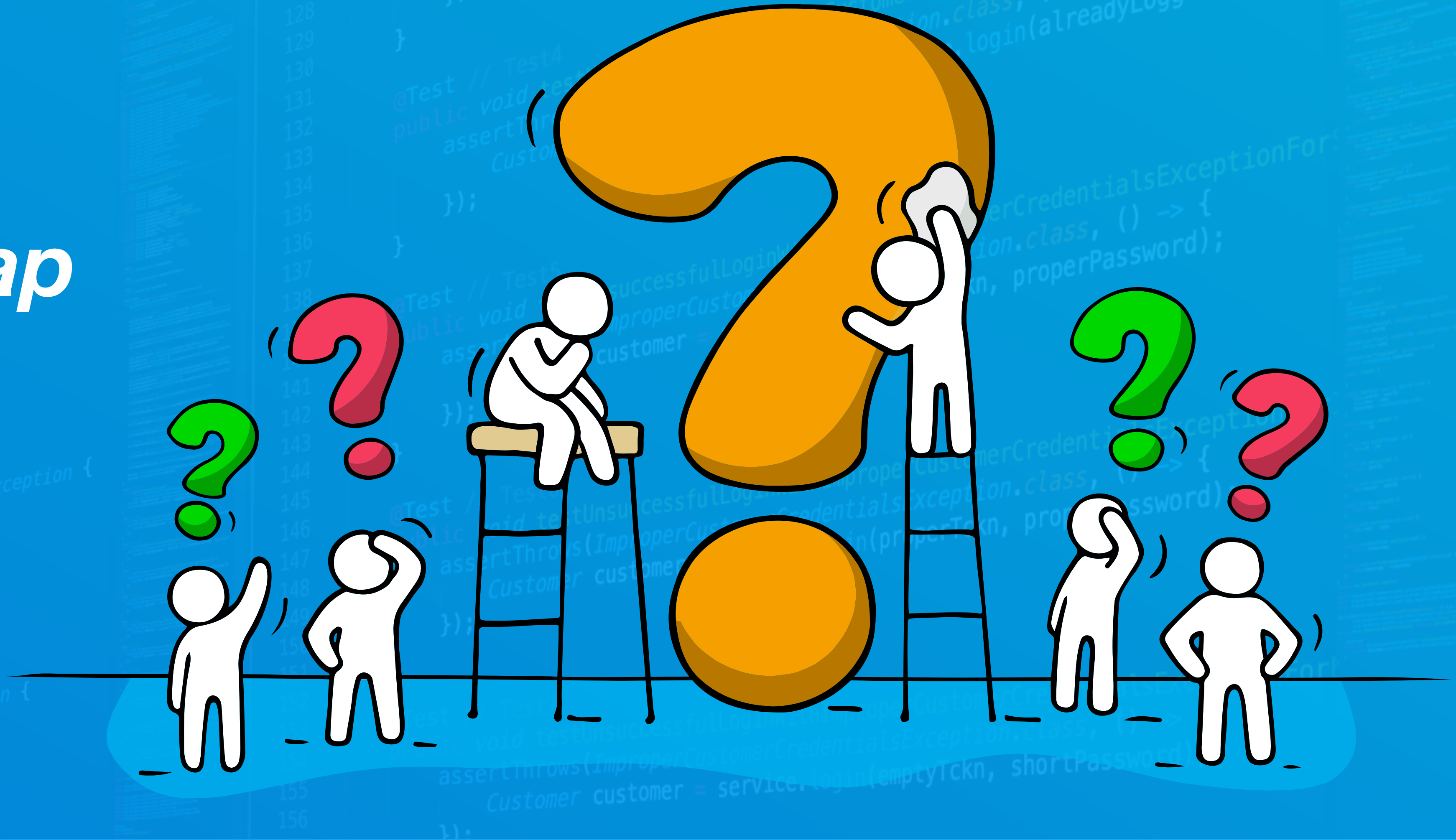
- Java'da nesnelerin oluşturulması, kodda geliştirici tarafından yapılır ama kendisine ihtiyaç kalmayan, kullanılmayan nesnelerin bellekten temizlenmesi, otomatik olarak, GC tarafından yapılır
- **Otomatik bellek yönetimi (automatic memory management)**
- C++'ta sadece nesnenin oluşturulması değil gerektiğinde bellekten silinmesi de geliştiricinin sorumluluğundadır
- Java'nın (hatta C# ve Go gibi dillerin) otomatik bellek yönetimi, bu anlamda bir ilerleme ve geliştiriciye bir kolaylıktır

Kullanılmayan Nesne



- Kullanılmayan ya da artık kendisine ihtiyaç kalmamış nesne ya da çöp (garbage) ne demektir?
- Eğer bir nesneyi gösteren hiç referans yoksa ya da o nesneyi gösteren tüm referanslar kapsam dışında (out of scope) kalmışlarsa, artık o nesneye ulaşılamaz
- Dolayısıyla o nesnenin bellekten silinmesi gerekir
- Çünkü bellek sonsuz değildir ve yeni nesneler için yer açılmalıdır

Soru ve Cevap Zamanı!





Bellek Sızıntısı

Bellek Sızıntısı (Memory Leak) - I



- Kendisine ihtiyaç kalmayan nesneler bellekten temizlenmezlerse birikerek
 - gittikçe artan bellek kullanımı, yani **bellek sızıntısı (memory leak)**
 - yeni nesnelerin oluşturulmasında yavaşlık
 - belleğin parçalanmışlığından (memory fragmentation) kaynaklanan büyük nesnelerin oluşturulmasında zorluk

gibi problemlere yol açarlar

OutOfMemoryError - I



- GC'nin bellekteki nesneleri toplayamamasının sebebi, nesnelerin referanslarının kapsam dışına çıkmaması ve bu yüzden bellekteki nesnelere her an ulaşma imkanının olmasıdır
- Yeni nesneler için heapte yer kalmadığında `java.lang.OutOfMemoryError` hatası fırlatılır
- `OutOfMemoryError` fırlatıldığında JVM çöker ve çalışması biter
- Aslında `OutOfMemoryError` yakalanabilir ama bunun pek bir anlamı yoktur

OutOfMemoryError - II

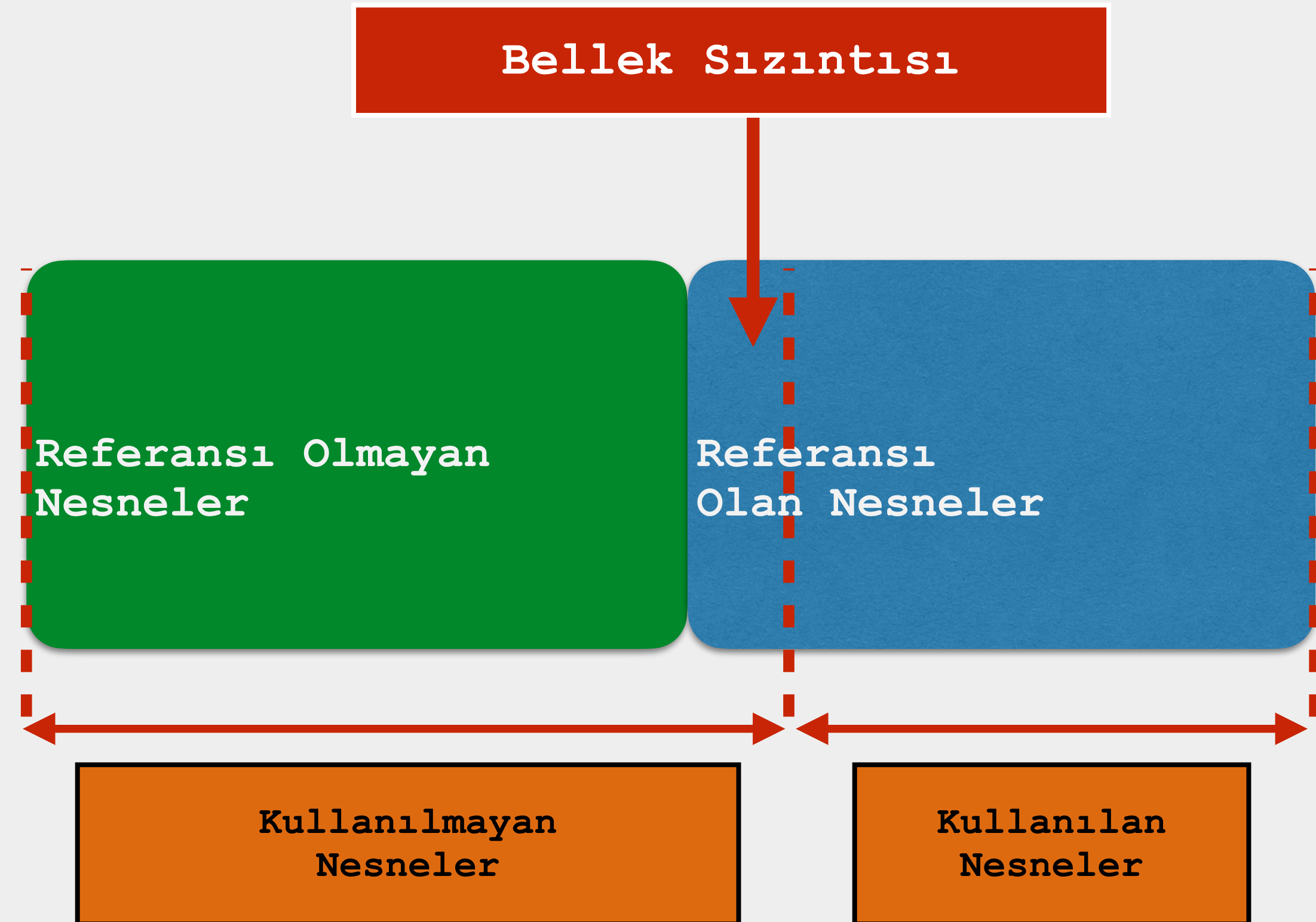


- Ne kadar hızlı nesne oluşturulursa oluşturulsun, toplanabilir oldukları müddetçe, GC daha etkin bir şekilde çalışarak bellek problemi yaşanmasını önler
- Problem, toplan(a)mayıp bellekte kalmaya devam eden nesnelerdir
- `java.lang.OutOfMemoryError` bellekte kalmak zorunda olan nesnelerden dolayı fırlatılır, GC'nin yavaş çalışmasından dolayı değil!

Bellek Sızıntısı (Memory Leak) - II



- Kendisine ulaşan hiçbir referans olmayan nesneler GC tarafından toplanır
- Uygulama içinde kullanılmadığı halde hâlâ kendisine referans olan nesneler ise, toplanması gerektiği halde, GC tarafından toplanamaz

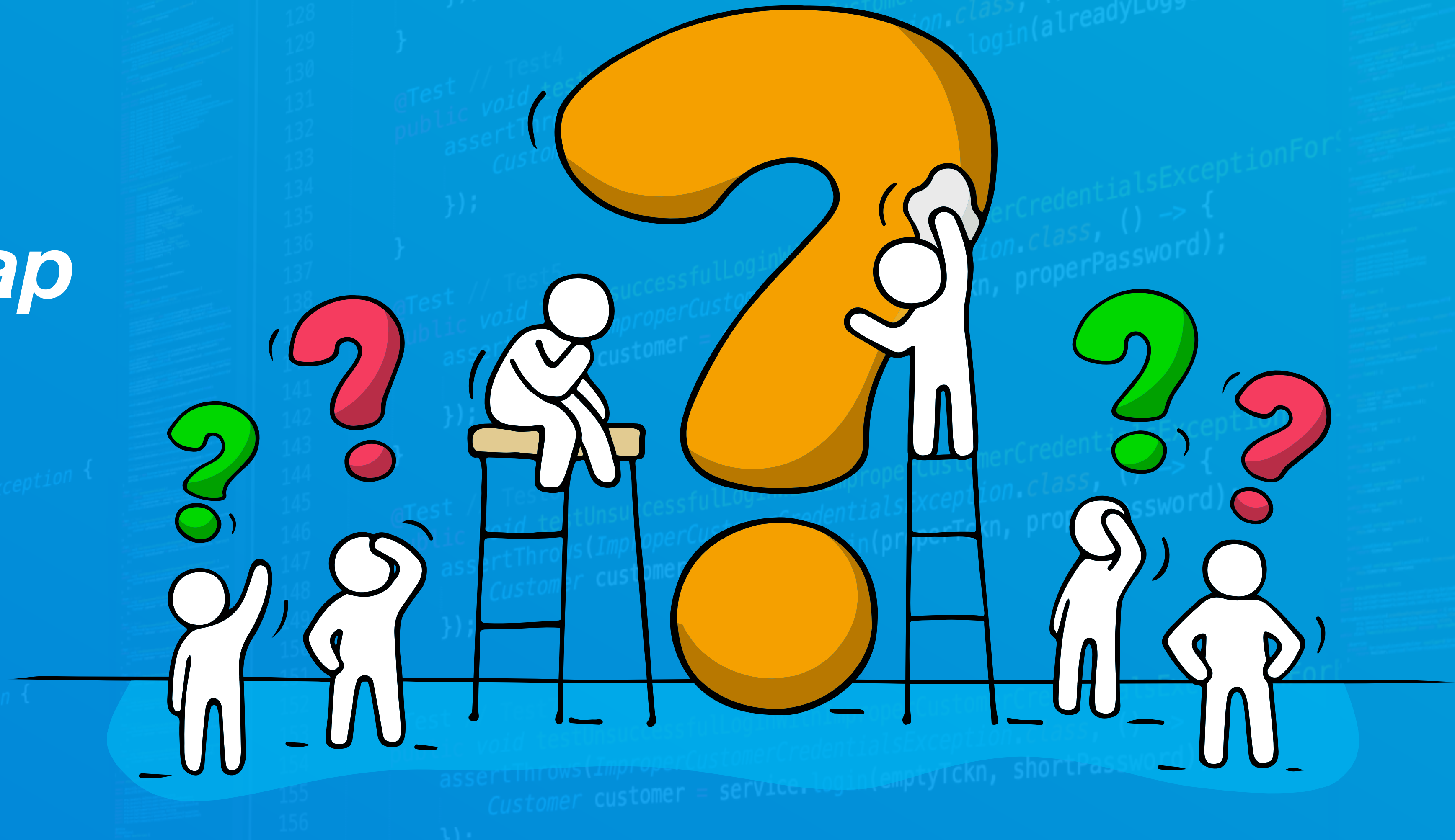


HeapDemo1



- HeapDemo1
- Örneğin 100_000_000 nesne oluşturacak şekilde çalıştırın ve sonucu gözlemleyin

Soru ve Cevap Zamanı!



Bellek Davranışı



Bellek Davranışı - I

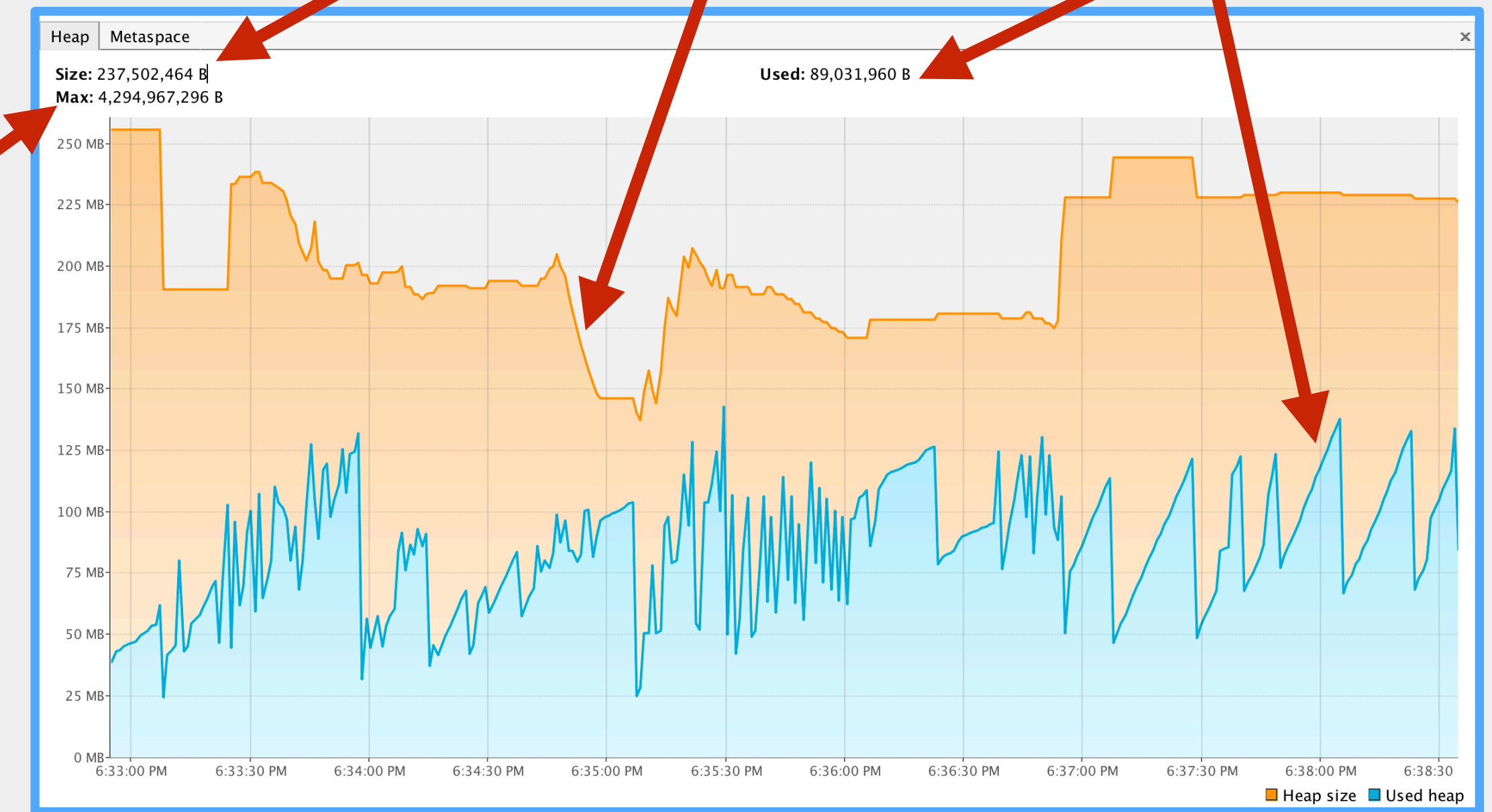


- Yanda nesne üreten ama nesnelerin hızlıca GC tarafından toplandığı bir uygulamanın kullandığı heap miktarının zamanla değişimi verilmiştir
- Turuncu çizgiler JVM'in sistemden aldığı toplam heap miktarını, mavi çizgiler ise kullanılan heap miktarını göstermektedir

Max heap

Anlık heap
büyüklüğü

Anlık kullanılan
heap miktarı

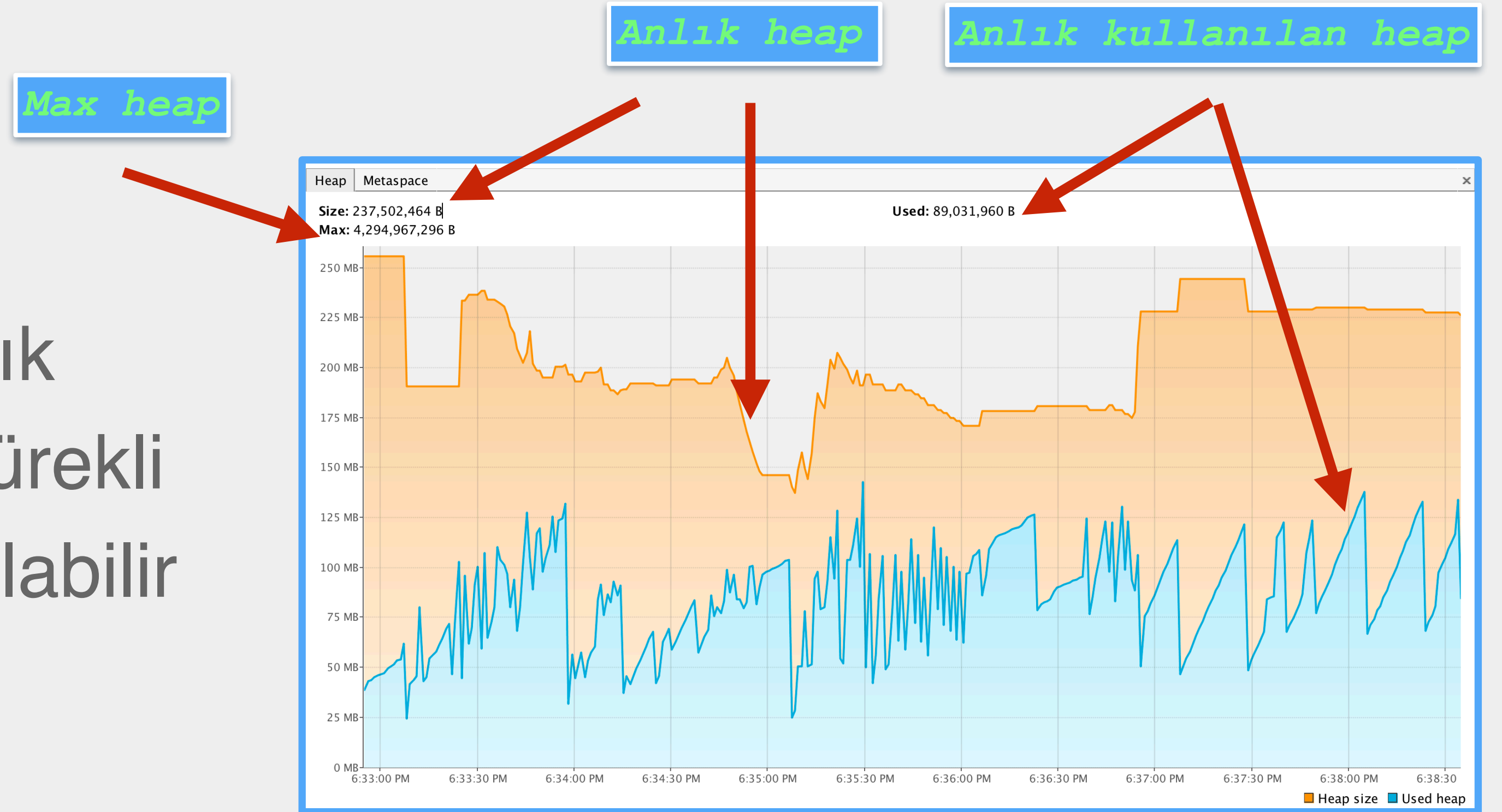


- Anlık heap büyüklüğü ile kullanılabilecek maksimum heap büyüklüğüne dikkat edin

Terimler - I



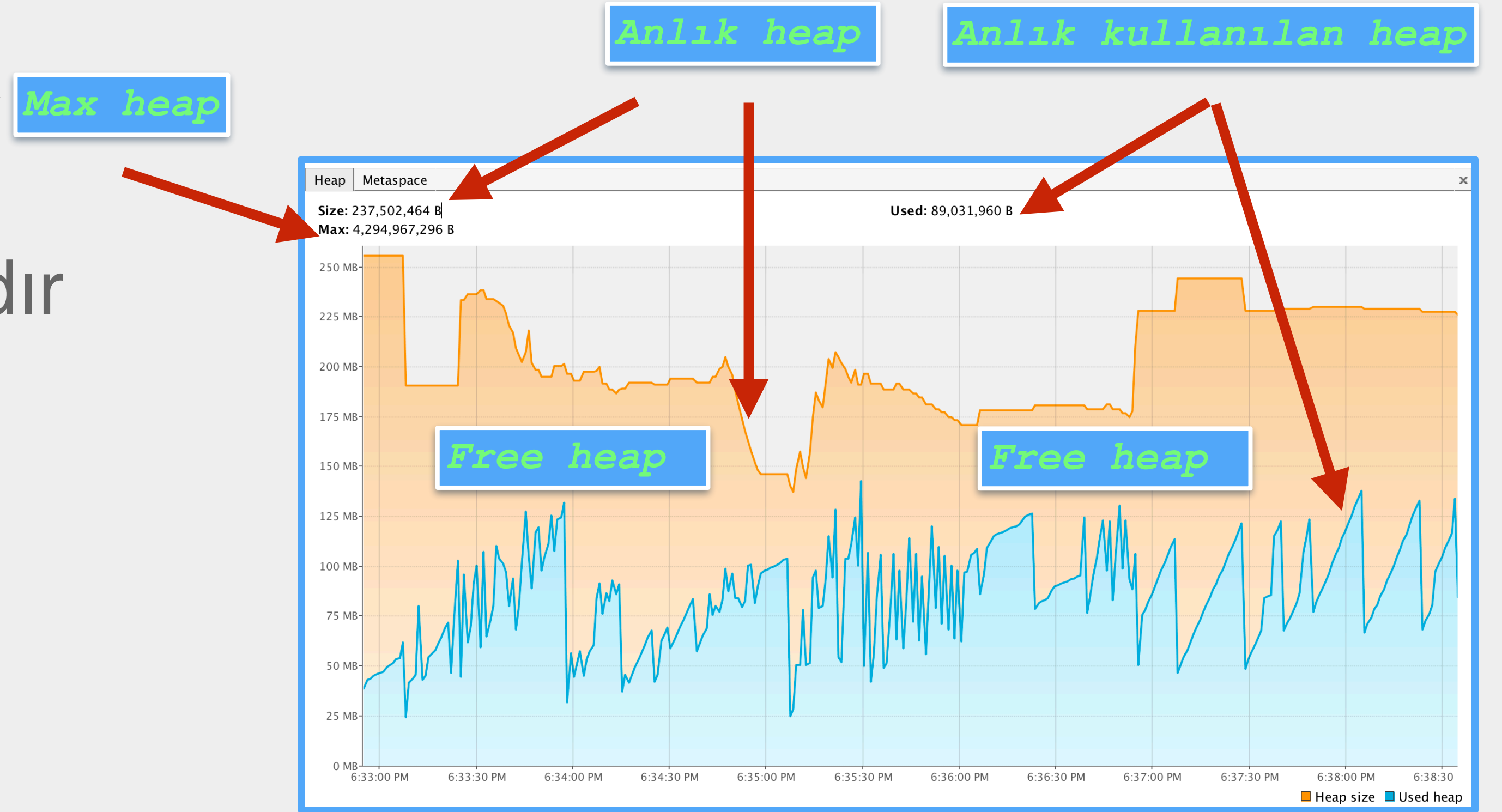
- **Max:** JVM'in kullanabileceği maksimum heap miktarıdır
- **Size:** JVM'e tahsis edilmiş anlık heap (total heap) miktarıdır; sürekli değişir ve en çok *max* kadar olabilir
- **Used:** Anlık kullanılan heap miktarıdır; sürekli değişir ve en çok *size* kadar olabilir



Terimler - II



- **Free:** Anlık *Size - Used* olarak hesaplanır ve geriye kalan, kullanılabilir bellek miktarıdır
- Şekilde kavuniçi renkli alandır



Bellek Davranışı - II



- Aşağıda çok bellek kullanan ve bu yüzden riske giren bir uygulamanın kullandığı heap miktarının zamanla değişimi verilmiştir

Anlık heap

Max heap

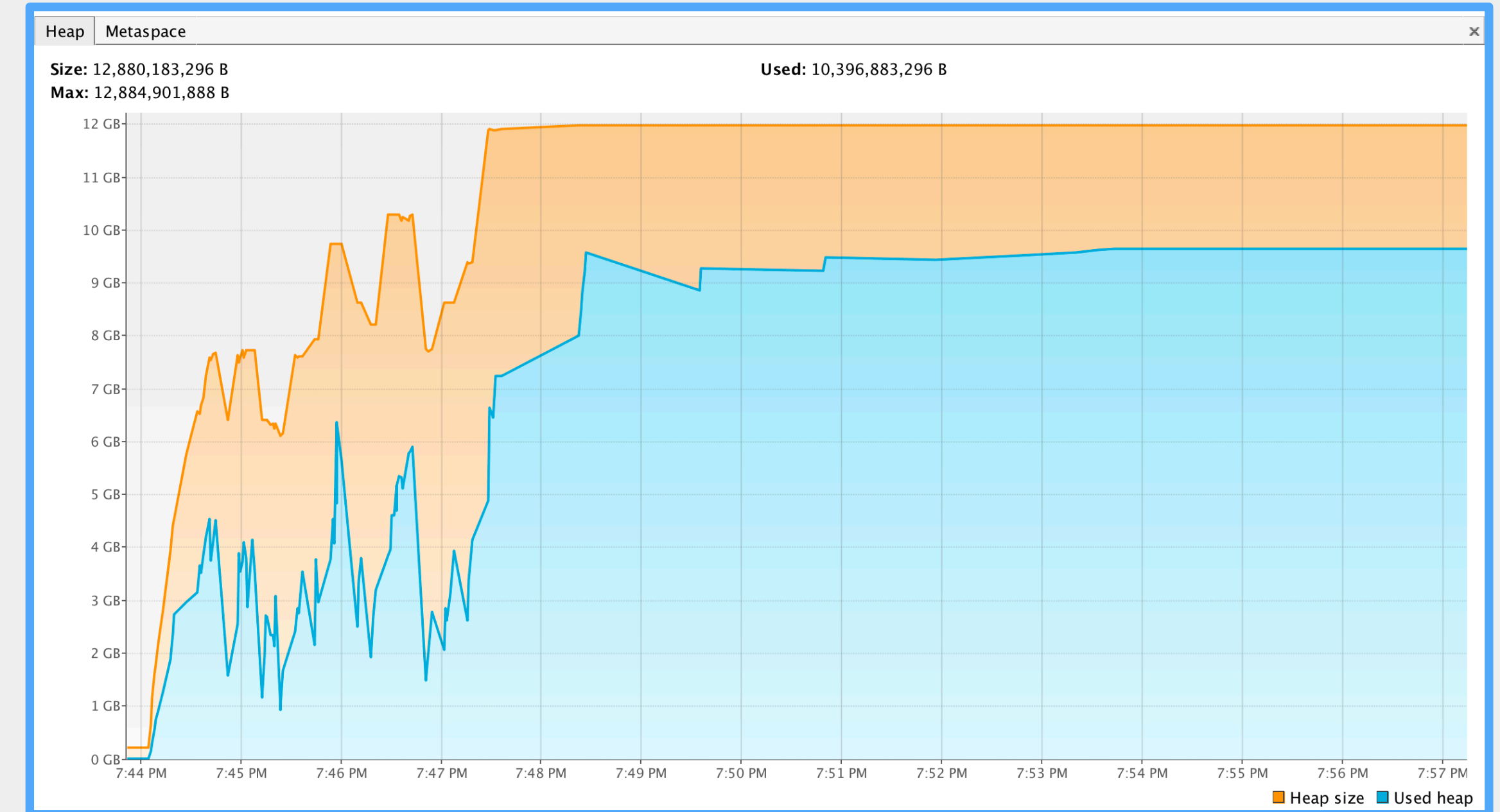


Anlık kullanılan heap

Bellek Davranışı - III



- Yanda bellek sızıntısından dolayı **OutOfMemoryError** alan bir uygulamanın kullandığı bellek miktarının değişimi verilmiştir
- Uygulamada başlarda nesneler sıklıkla toplanmasına rağmen sonrasında GC'nin nesneleri toplayamamasından dolayı JVM hata olarak çalışmasına son vermiştir



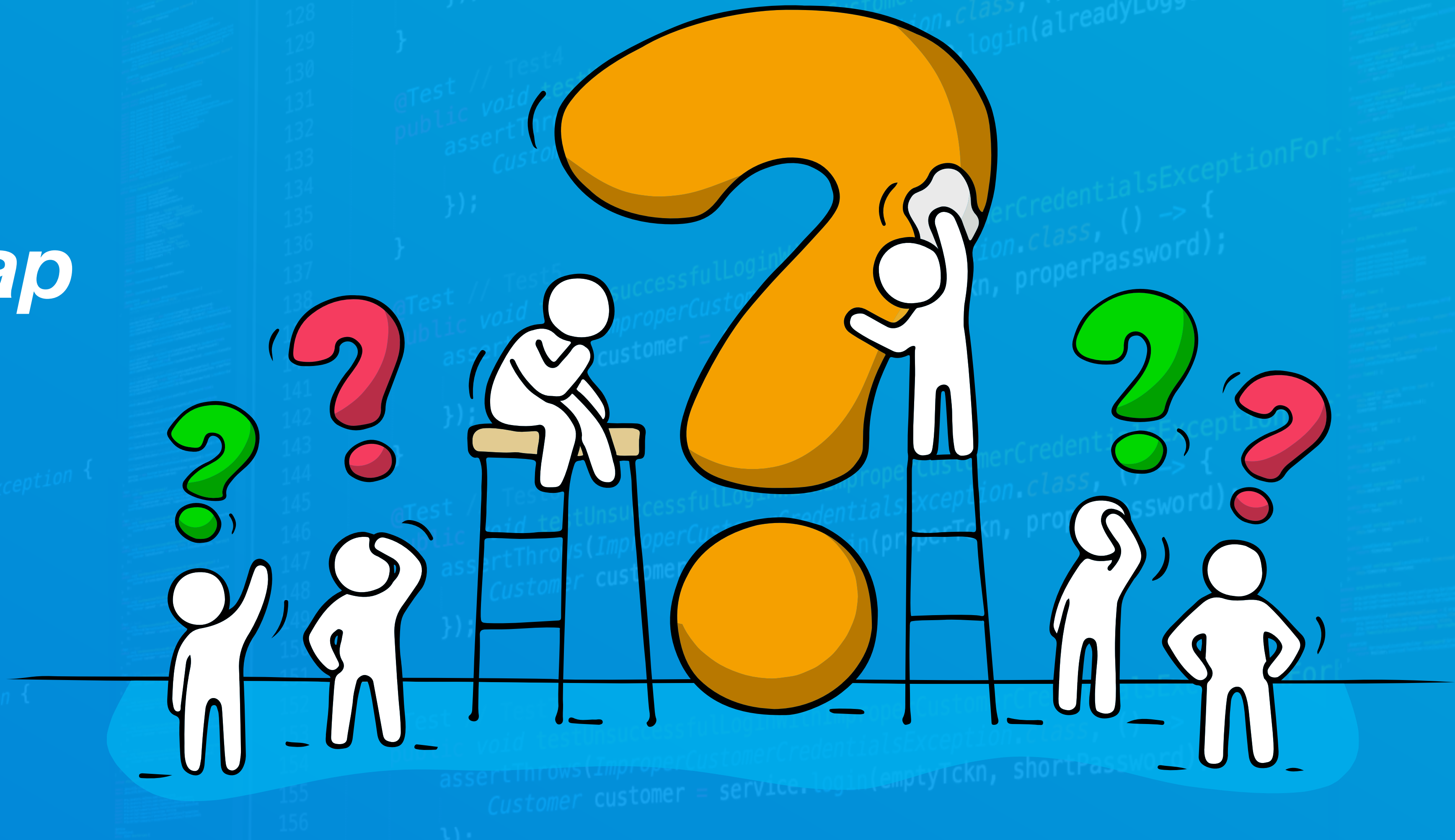
```
Exception in thread "main" java.lang.OutOfMemoryError: GC overhead limit exceeded
    at java.util.Arrays.copyOfRange(Arrays.java:3664)
    at java.lang.String.<init>(String.java:207)
    at java.lang.StringBuilder.toString(StringBuilder.java:407)
    at org.javaturk.performance.memory.HeapDemo2.run2(HeapDemo2.java:72)
    at org.javaturk.performance.memory.HeapDemo2.main(HeapDemo2.java:47)
```

HeapDemo2



- HeapDemo2
 - Önce `run1 ()` sonra da `run2 ()` metodunu 100_000_000 nesne oluşturacak şekilde çalıştırın ve sonucu gözlemleyin

Soru ve Cevap Zamanı!





Etkin Bellek Yönetimi

Etkin Bellek Yönetimi - I



- Bellek sızıntılarının önüne geçmek için şunlar yapılabilir:
 - Gereksiz nesne oluşturmaktan kaçınmak
 - Nesneleri olabildiğince kısa ömürlü tutmak

Etkin Bellek Yönetimi - II



- Nesneleri kısa ömürlü tutmak için
 - Gerekmiyorsa nesnenin referanslarını metotlara ve dizi gibi torbalara geçmemek
 - Bir nesneye ne kadar çok referans oluşturulursa bellekte kalma ihtimali o kadar artar
 - Bir referans bir metoda geçildiğinde orada o referansa ne yapıldığı bilinemez; dolayısıyla o referansın gösterdiği nesnenin bellekte kalmaya devam etme ihtimali artar

Etkin Bellek Yönetimi - II



- Yaşayan nesnelere ihtiyaç kalmadığında referanslarını kapsam dışı yapmak ya da nesne ile ilgisini kesmek
- Bunun için nesnenin referanslarına **null** atamak yardımcı olur

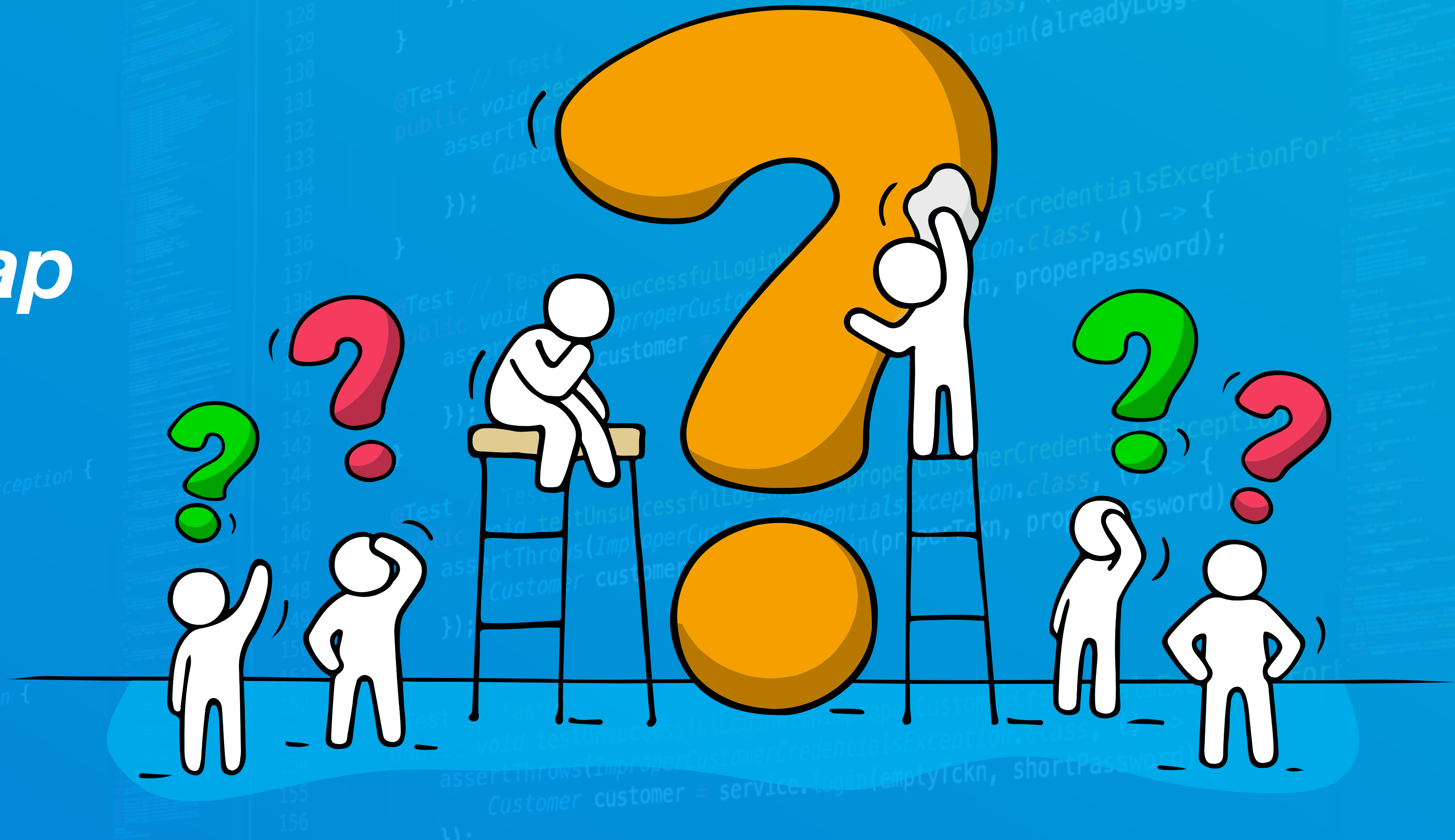
Nesne-Referans İlişkisini Kesmek



- GC ulaşılabilir nesneleri işaretler; işaretlenmeyenler toplanır
- Dolayısıyla referansları `null` yapılan nesneler de GC tarafından toplanabilir

```
// Imagine following code takes longer  
// to execute causing Car object to live even  
// though it is not needed anymore.  
Car car = new Car();  
car.accelerate(120);  
...  
car = null; // Release the reference  
           // so the Car object can be GCed.
```

Soru ve Cevap Zamanı!





Garbage Collection

Çöp Ne Zaman Toplanır?



- Çöp toplama (Garbage Collection), çöp toplayıcının (Garbage Collector) inisiyatifindedir
- Bu inisiyatifi etkileyen pek çok etmen vardır:
 - Kalan bellek miktarı GC'nin çöp toplamaya başlamasına sebep olabilir
- JVM'in heap için min ve max değerleri gibi bellek ve GC ile ilgili pek çok parametresi vardır
 - Genelde JVM başlatılırken bu parametreler verilebilir

GC Çağrısı



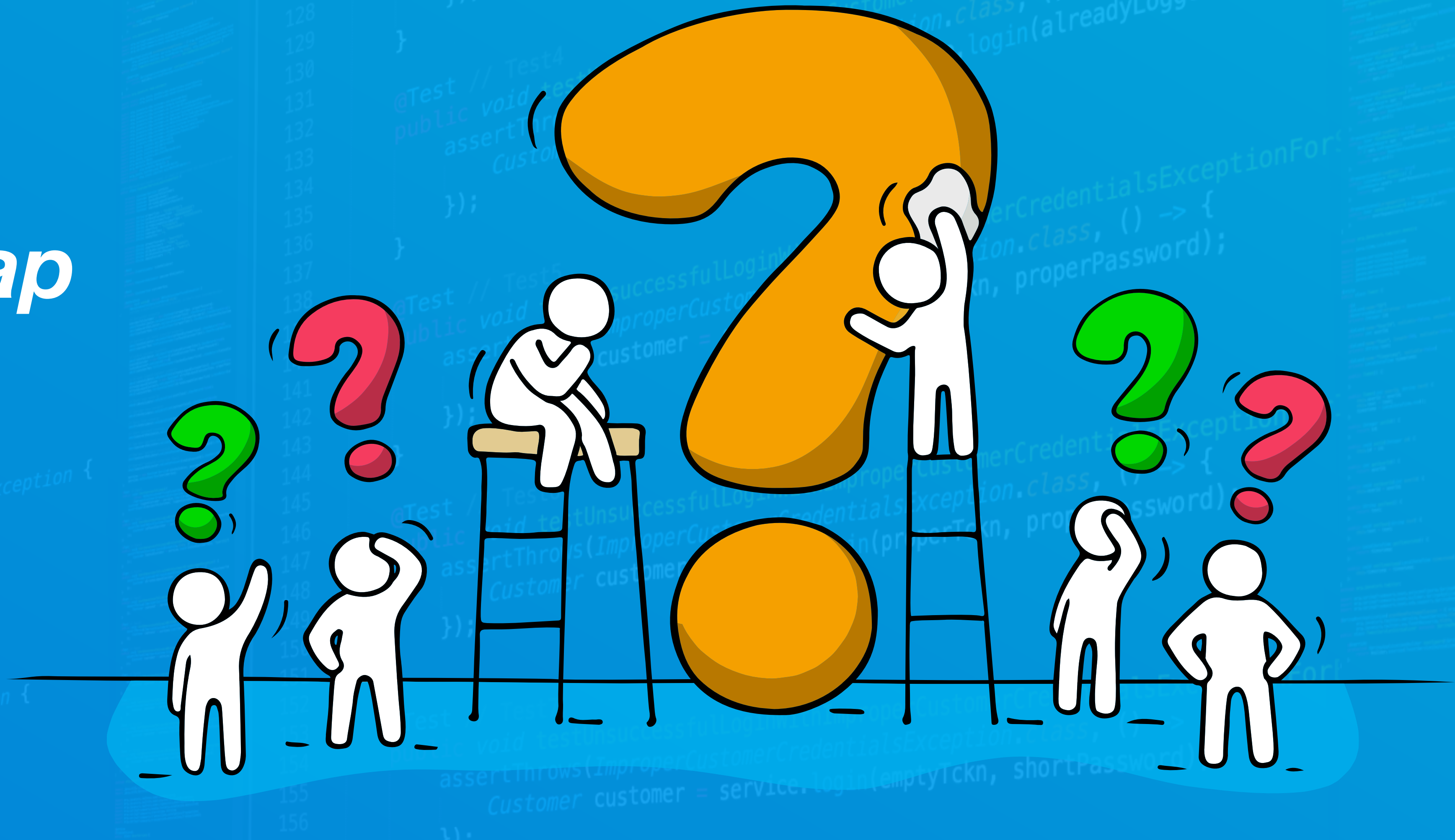
- `System.gc()` , GC'yi temizlik yapmak üzere uyarı veren bir metot çağrısıdır
- `System.gc()` , sadece bir uyarıdır ve GC temizlik yapmak zorunda değildir; GC'nin ayrı bir karar mekanizması vardır
- Ayrıca JVM, bir parametre ile `System.gc()` 'nin GC'yi uyarmasını engelleyebilir
- Dolayısıyla sık `System.gc()` kullanımı önerilmez

GC'nin Maliyeti



- Ayrıca GC'nin kullanılmayan nesneleri toplaması bedava değildir
 - GC çalışırken genel olarak uygulama durur; buna **stop-the-world, STW**, denir
- Dolayısıyla JVM'in kullandığı heap miktarını arttırmak, bellek sızıntılarını önlemek ya da `OutOfMemoryError` almamak için iyi bir yöntem değildir
- Ayrıca kullanılan heap ne kadar büyükse nesneleri temizlemek o kadar zaman alır

Soru ve Cevap Zamanı!



Ödevler





1. **MemoryUtil**'e benzer bir şekilde çalışan ve heap'in max, total, free ve used bellek değerlerini ayrı ayrı hesaplayan ve yazdıran metotlara sahip **MemoryProbe** isimli bir sınıf yazın
 - a. **MemoryUtil**'de used değerinin nasıl hesaplandığına dikkat edin



2. HeapDemo1 ile HeapDemo2'nin `run1 ()` metodunu aynı nesne sayısı ile çalıştırın
- a. Birinde *Used* düz seyrederken diğerinde neden sürekli artıyor?
 - b. Cevabı, koddaki tek bir satırı göstererek açıklayın
 - c. Bu satır, slayt 15'teki dört kümeden hangisinin içine düşüyor?



3. Büyük bir nesneyi `static` bir alanda tutun ve *Used* değerini yazdırın. Sonra alana `null` atayıp tekrar yazdırın. Ardından `System.gc()` çağırıp bir kez daha yazdırın
 - a. Üç ölçümü karşılaştırın. `null` atamak tek başına belleği boşalttı mı?
 - b. Slayt 30 "GC ulaşılabilir nesneleri işaretler; işaretlenmeyenler toplanır" diyor. Ölçtüğünüz üç değer bu cümleyi nasıl doğruluyor?

Bundan Sonra?



Bundan Sonra?



- Bundan sonra kod organizasyonunu ele alacağız

Bölüm Sonu

Soru ve Cevap Zamanı!

